

Study Guide: Triggers in SQL

Andrés Oswaldo Calderón Romero, PhD.

May 18, 2026

1 Introduction to Triggers

A **trigger** is a database object that automatically executes a predefined action in response to certain events on a table or view. It is a type of stored procedure that is triggered automatically when a specified database operation occurs.

Key Features

- Automated execution
- Event-driven
- Typically used for enforcing business rules, maintaining audit trails, and validating data

2 Why Use Triggers?

Triggers are particularly useful for:

- **Data Integrity:** Enforcing complex constraints that cannot be captured by primary keys, unique constraints, or check constraints.
- **Audit Trails:** Automatically recording changes to critical data.
- **Business Rules:** Implementing complex logic at the database level.
- **Synchronization:** Keeping related tables consistent.
- **Security:** Validating data before changes are made.

3 Types of Triggers

Triggers can be classified based on the timing and event that triggers their execution:

Timing

- **BEFORE Trigger:** Executes before the triggering event.
- **AFTER Trigger:** Executes after the triggering event.
- **INSTEAD OF Trigger:** Replaces the triggering event (typically for views).

Event

- **INSERT Trigger:** Fires when a new row is inserted.
- **UPDATE Trigger:** Fires when an existing row is updated.
- **DELETE Trigger:** Fires when a row is deleted.

Nested Triggers

A **nested trigger** is a trigger that is fired as a result of another trigger. This occurs when the execution of one trigger causes a data modification (INSERT, UPDATE, or DELETE) that activates a second trigger on the same or a different table.

Key Characteristics

- **Chained Execution:** Triggers can form a chain where one trigger's action initiates another.
- **Depth Control:** Some database systems, like SQL Server, allow you to limit the nesting level to prevent infinite loops.
- **Performance Impact:** Excessive nesting can lead to performance issues and complex debugging.
- **Transactional Context:** Nested triggers operate within the same transaction, meaning a failure in a nested trigger can roll back the entire chain.

Example: Nested Trigger in PostgreSQL

This example demonstrates how a trigger on one table can cause a second trigger on a related table to activate.

```
1  -- Table for tracking customer status changes
2  CREATE TABLE customer_status_changes (
3      change_id SERIAL PRIMARY KEY,
4      customer_id INT,
5      old_status VARCHAR(50),
6      new_status VARCHAR(50),
7      change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
8  );
9
10 -- Table for logging audit history
11 CREATE TABLE audit_log (
12     log_id SERIAL PRIMARY KEY,
13     message TEXT,
14     logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```

15 );
16
17 -- Trigger function to log status changes
18 CREATE OR REPLACE FUNCTION log_status_changes()
19 RETURNS TRIGGER AS $$
20 BEGIN
21     INSERT INTO customer_status_changes (customer_id, old_status, new_status)
22     VALUES (OLD.customer_id, OLD.status, NEW.status);
23     RETURN NEW;
24 END;
25 $$ LANGUAGE plpgsql;
26
27 -- Trigger function for nested logging
28 CREATE OR REPLACE FUNCTION log_audit_message()
29 RETURNS TRIGGER AS $$
30 BEGIN
31     INSERT INTO audit_log (message)
32     VALUES ('Customer status updated for customer ID: ' || NEW.customer_id);
33     RETURN NEW;
34 END;
35 $$ LANGUAGE plpgsql;
36
37 -- Primary trigger on customer status updates
38 CREATE TRIGGER customer_status_trigger
39 AFTER UPDATE ON customers
40 FOR EACH ROW
41 EXECUTE FUNCTION log_status_changes();
42
43 -- Nested trigger on customer status change table
44 CREATE TRIGGER nested_audit_trigger
45 AFTER INSERT ON customer_status_changes
46 FOR EACH ROW
47 EXECUTE FUNCTION log_audit_message();

```

4 Basic Syntax of Triggers (PostgreSQL)

```

1 CREATE TRIGGER trigger_name
2 { BEFORE | AFTER | INSTEAD OF } { INSERT | UPDATE | DELETE }
3 ON table_name
4 FOR EACH { ROW | STATEMENT }
5 WHEN (condition)
6 EXECUTE FUNCTION function_name();

```

5 Practical Examples

Audit Log Trigger

Tracking changes in an employees table:

```

1 CREATE TABLE employees_audit (

```

```

2     audit_id SERIAL PRIMARY KEY,
3     employee_id INT,
4     operation VARCHAR(10),
5     change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6 );
7
8 CREATE OR REPLACE FUNCTION log_employee_changes()
9 RETURNS TRIGGER AS $$
10 BEGIN
11     INSERT INTO employees_audit (employee_id, operation)
12     VALUES (NEW.employee_id, TG_OP);
13     RETURN NEW;
14 END;
15 $$ LANGUAGE plpgsql;
16
17 CREATE TRIGGER employee_audit_trigger
18 AFTER INSERT OR UPDATE OR DELETE
19 ON employees
20 FOR EACH ROW
21 EXECUTE FUNCTION log_employee_changes();

```

Data Validation Trigger

Preventing negative salaries in the `employees` table:

```

1 CREATE OR REPLACE FUNCTION validate_salary()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NEW.salary < 0 THEN
5         RAISE EXCEPTION 'Salary cannot be negative';
6     END IF;
7     RETURN NEW;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER salary_check
12 BEFORE INSERT OR UPDATE
13 ON employees
14 FOR EACH ROW
15 EXECUTE FUNCTION validate_salary();

```

6 Transition Tables in Triggers

A **transition table** in PostgreSQL is a special table used in triggers to capture the full set of affected rows during bulk operations. Unlike the `OLD` and `NEW` row-level references, transition tables provide a set-based view of all modified rows, making them ideal for performance tuning and bulk data processing. They are available for `AFTER` triggers on `UPDATE` and `DELETE` operations.

- Useful for auditing large changes without per-row overhead.
- Enables set-based processing within triggers for better efficiency.

- Can be referenced in the trigger function using the `REFERENCING` clause.

Example: Using Transition Tables for Bulk Auditing

This example demonstrates how to use transition tables to log bulk updates to an `employees` table:

```
1 CREATE TABLE employees_bulk_audit (  
2     audit_id SERIAL PRIMARY KEY,  
3     employee_id INT,  
4     old_salary NUMERIC,  
5     new_salary NUMERIC,  
6     change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
7 );  
8  
9 CREATE OR REPLACE FUNCTION log_bulk_salary_changes()  
10 RETURNS TRIGGER AS $$  
11 BEGIN  
12     INSERT INTO employees_bulk_audit (employee_id, old_salary, new_salary)  
13     SELECT OLD.employee_id, OLD.salary, NEW.salary  
14     FROM OLD_TABLE AS OLD, NEW_TABLE AS NEW  
15     WHERE OLD.employee_id = NEW.employee_id;  
16     RETURN NULL;  
17 END;  
18 $$ LANGUAGE plpgsql;  
19  
20 CREATE TRIGGER bulk_salary_audit  
21 AFTER UPDATE ON employees  
22 REFERENCING OLD TABLE AS OLD_TABLE NEW TABLE AS NEW_TABLE  
23 FOR EACH STATEMENT  
24 EXECUTE FUNCTION log_bulk_salary_changes();
```

7 Best Practices for Using Triggers

- Use triggers sparingly to avoid performance issues.
- Document the purpose of each trigger clearly.
- Avoid complex logic that can make debugging difficult.
- Use triggers for auditing and validation, but not as a primary business logic layer.
- Test triggers extensively before deploying to production.
- Use transition tables for bulk operations to reduce overhead.
- Monitor trigger performance regularly to avoid bottlenecks.

8 Additional Content

- “*SQL Triggers: A Beginner’s Guide*”. Oluseye Jeremiah. 2024. Link in [DataCamp](#). [3].

- “*Triggers In SQL— Triggers In Database, SQL Triggers Tutorial For Beginners*”. Edureka!, 2023. Link in [YouTube](#). [1].
- “*¿Qué diablos es un Trigger? Ejemplo sencillo en Sql Server*”. Héctor de León. 2018. In Spanish. Link in [YouTube](#). [2].
- *Procedural Programming with PostgreSQL PL/pgSQL: Design Complex Database-Centric Applications with PL/pgSQL*. Baji Shaik and Dinesh Kumar Chemuduru. 2023. Apress. [4]. In particular Chapters 1, 2 and 3.

9 Exercises & Solutions

Below, you will find a selection of illustrative exercises along with their corresponding solutions. We highly encourage you to take a moment to work through each problem **on your own** before checking the answers. While the solutions are readily available, you will gain a much deeper understanding of the core concepts by tackling the logic yourself first.

1. Create a trigger to prevent the deletion of VIP customers.
2. Implement a trigger that logs every update to a `products` table, including the old and new price.
3. Write a trigger that automatically updates a stock table whenever a sale is made.
4. Create a trigger that validates email format before inserting a customer record.
5. Optimize a trigger for bulk data updates using transition tables.
6. Design a trigger that automatically archives deleted rows into a history table.
7. Implement a performance monitoring trigger to track long-running updates.

Solutions

Exercise 1: Preventing the Deletion of VIP Customers

To prevent the deletion of VIP customers, you can create a trigger that raises an exception if an attempt is made to delete a customer marked as VIP.

```
1 CREATE OR REPLACE FUNCTION prevent_vip_deletion()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF OLD.is_vip THEN
5         RAISE EXCEPTION 'Cannot delete VIP customers.';
6     END IF;
7     RETURN OLD;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER vip_deletion_blocker
12 BEFORE DELETE ON customers
13 FOR EACH ROW
14 EXECUTE FUNCTION prevent_vip_deletion();
```

Exercise 2: Logging Product Price Updates

To log every price update in a `products` table, you can use a trigger that captures both the old and new prices.

```
1 CREATE TABLE product_price_log (
2     log_id SERIAL PRIMARY KEY,
3     product_id INT,
4     old_price NUMERIC,
5     new_price NUMERIC,
6     change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );
8
9 CREATE OR REPLACE FUNCTION log_price_changes()
10 RETURNS TRIGGER AS $$
11 BEGIN
12     INSERT INTO product_price_log (product_id, old_price, new_price)
13     VALUES (OLD.product_id, OLD.price, NEW.price);
14     RETURN NEW;
15 END;
16 $$ LANGUAGE plpgsql;
17
18 CREATE TRIGGER price_update_logger
19 AFTER UPDATE ON products
20 FOR EACH ROW
21 EXECUTE FUNCTION log_price_changes();
```

Exercise 3: Stock Update on Sale

To automatically update the stock of a product when a sale is made, you can use a trigger to adjust the inventory.

```
1 CREATE OR REPLACE FUNCTION update_stock_on_sale()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     UPDATE products
5     SET stock = stock - NEW.quantity
6     WHERE product_id = NEW.product_id;
7     RETURN NEW;
8 END;
9 $$ LANGUAGE plpgsql;
10 CREATE TRIGGER stock_update_trigger
11 AFTER INSERT ON sales
12 FOR EACH ROW
13
14 EXECUTE FUNCTION update_stock_on_sale();
```

Exercise 4: Email Format Validation

To validate email formats before inserting customer records, you can create a trigger to enforce proper formatting.

```
1 CREATE OR REPLACE FUNCTION validate_email_format()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NOT NEW.email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$' THEN
5         RAISE EXCEPTION 'Invalid email format: %', NEW.email;
6     END IF;
7     RETURN NEW;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER email_validation_trigger
12 BEFORE INSERT OR UPDATE ON customers
13 FOR EACH ROW
14 EXECUTE FUNCTION validate_email_format();
```

Exercise 5: Bulk Data Updates Using Transition Tables

To efficiently handle bulk data updates, you can use transition tables to capture the old and new state of rows during a single operation.

```
1 CREATE TABLE bulk_update_audit (
2     audit_id SERIAL PRIMARY KEY,
3     product_id INT,
```



```

4      old_price NUMERIC,
5      new_price NUMERIC,
6      change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7  );
8
9  CREATE OR REPLACE FUNCTION log_bulk_price_changes()
10 RETURNS TRIGGER AS $$
11 BEGIN
12     INSERT INTO bulk_update_audit (product_id, old_price, new_price)
13     SELECT OLD.product_id, OLD.price, NEW.price
14     FROM OLD_TABLE AS OLD
15     JOIN NEW_TABLE AS NEW
16     ON OLD.product_id = NEW.product_id;
17     RETURN NULL;
18 END;
19 $$ LANGUAGE plpgsql;
20
21 CREATE TRIGGER bulk_price_update_trigger
22 AFTER UPDATE ON products
23 REFERENCING OLD TABLE AS OLD_TABLE NEW TABLE AS NEW_TABLE
24 FOR EACH STATEMENT
25 EXECUTE FUNCTION log_bulk_price_changes();

```

Exercise 6: Archiving Deleted Rows

To automatically archive deleted rows, you can create a trigger that stores deleted data in a history table.

```

1  CREATE TABLE customer_archive (
2      archive_id SERIAL PRIMARY KEY,
3      customer_id INT,
4      name VARCHAR(255),
5      email VARCHAR(255),
6      deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7  );
8
9  CREATE OR REPLACE FUNCTION archive_deleted_customers()
10 RETURNS TRIGGER AS $$
11 BEGIN
12     INSERT INTO customer_archive (customer_id, name, email)
13     VALUES (OLD.customer_id, OLD.name, OLD.email);
14     RETURN OLD;
15 END;
16 $$ LANGUAGE plpgsql;
17
18 CREATE TRIGGER archive_on_delete
19 BEFORE DELETE ON customers
20 FOR EACH ROW
21 EXECUTE FUNCTION archive_deleted_customers();

```

Exercise 7: Performance Monitoring

To track long-running updates, you can create a trigger to log slow transactions for performance analysis.

```

1 CREATE TABLE slow_query_log (
2     log_id SERIAL PRIMARY KEY,
3     table_name VARCHAR(255),
4     operation VARCHAR(10),
5     duration_ms INT,
6     logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );
8
9 CREATE OR REPLACE FUNCTION log_slow_updates()
10 RETURNS TRIGGER AS $$
11 DECLARE
12     start_time TIMESTAMP;
13     end_time TIMESTAMP;
14     duration INT;
15 BEGIN
16     start_time := clock_timestamp();
17     -- Simulate some processing delay for demonstration
18     PERFORM pg_sleep(0.5);
19     end_time := clock_timestamp();
20     duration := EXTRACT(MILLISECOND FROM end_time - start_time);
21
22     IF duration > 500 THEN
23         INSERT INTO slow_query_log (table_name, operation, duration_ms)
24         VALUES (TG_TABLE_NAME, TG_OP, duration);
25     END IF;
26
27     RETURN NEW;
28 END;
29 $$ LANGUAGE plpgsql;
30
31 CREATE TRIGGER slow_update_logger
32 AFTER UPDATE OR DELETE ON customers
33 FOR EACH ROW
34 EXECUTE FUNCTION log_slow_updates();

```

References

- [1] Edureka! Triggers in sql — triggers in database — sql triggers tutorial for beginners, 2023. Accessed: 2025-05-19.
- [2] hdeleon.net. ¿qué diablos es un trigger? — ejemplo sencillo en sql server, 2018. Accessed: 2025-05-19.
- [3] Oluseye Jeremiah. Sql triggers: A beginner's guide, 2024. Accessed: 2025-05-19.
- [4] Baji Shaik and Dinesh Kumar Chemuduru. *Procedural Programming with PostgreSQL PL/pgSQL: Design Complex Database-Centric Applications with PL/pgSQL*. Apress, Texas, TX, USA, 1 edition, 2023.